CSE 645 Project 1

Group Members	
Name	**UCID**
Kurudunje Deekshith Shetty	Ks2378
Yahya Sheikh	Ys62

Problem 1:

Part 1: Code uploaded on Canvas

Answer:

user0:williamsburg
user1:wisconsin
user2:sheffield
user3:oceanography
user4:rachmaninoff
user5:hawaii
user6:alicia
user7:academia
user8:marietta
user9:napoleon

Part 2: Code uploaded on Canvas

Answer:

user1:zmodem
user0:nepenthe
user9:anthropogenic
user5:yellowstone

Part 3:

Password Found for user4:darkchocolate

In my quest to crack the fifth common password following part 2 of the same question where we found the passwords for user0, user1, user5 and user9 as nepenthe, zmodem, yellowstone and anthropogenic respectively with the help of the common-passwords.txt file provided, I hit some roadblocks. Despite trying various password lists, I could not find the right one. Then, I decided to be smarter about it.

I tried the most common passwords for windows, the top ten thousand most used passwords In Linux, Dark web Most used passwords list and many other famous dictionaries like common corporate passwords, NCSC 100k most common passwords, etc. but could not find it in them.

Instead of blindly guessing, I took a closer look at the passwords I had already cracked. They were all alphabetic and less than 16 characters long. Armed with this insight, I focused my search on similar patterns.

The rockyou.txt dictionary seemed promising, but it was huge! To save time, I deleted out the passwords I already found in the shadow file and only looked at shorter ones. This narrowed down the options and helped me crack the elusive fifth password. The password for user4 which is **darkchocolate**.

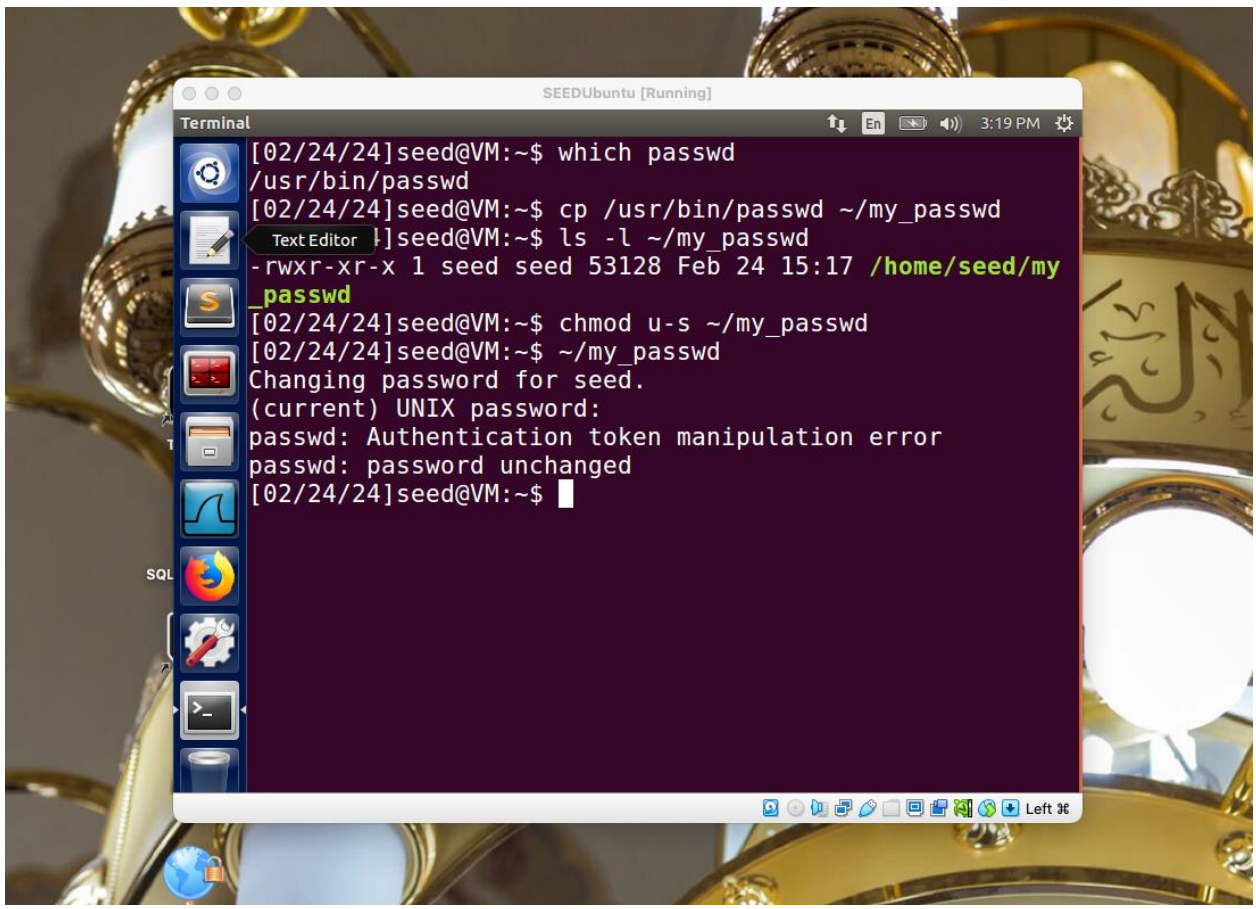
In the end, it was all about being strategic and adapting my approach. And it paid off!

The URL for the Dictionary file is:
https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt

Problem 2:

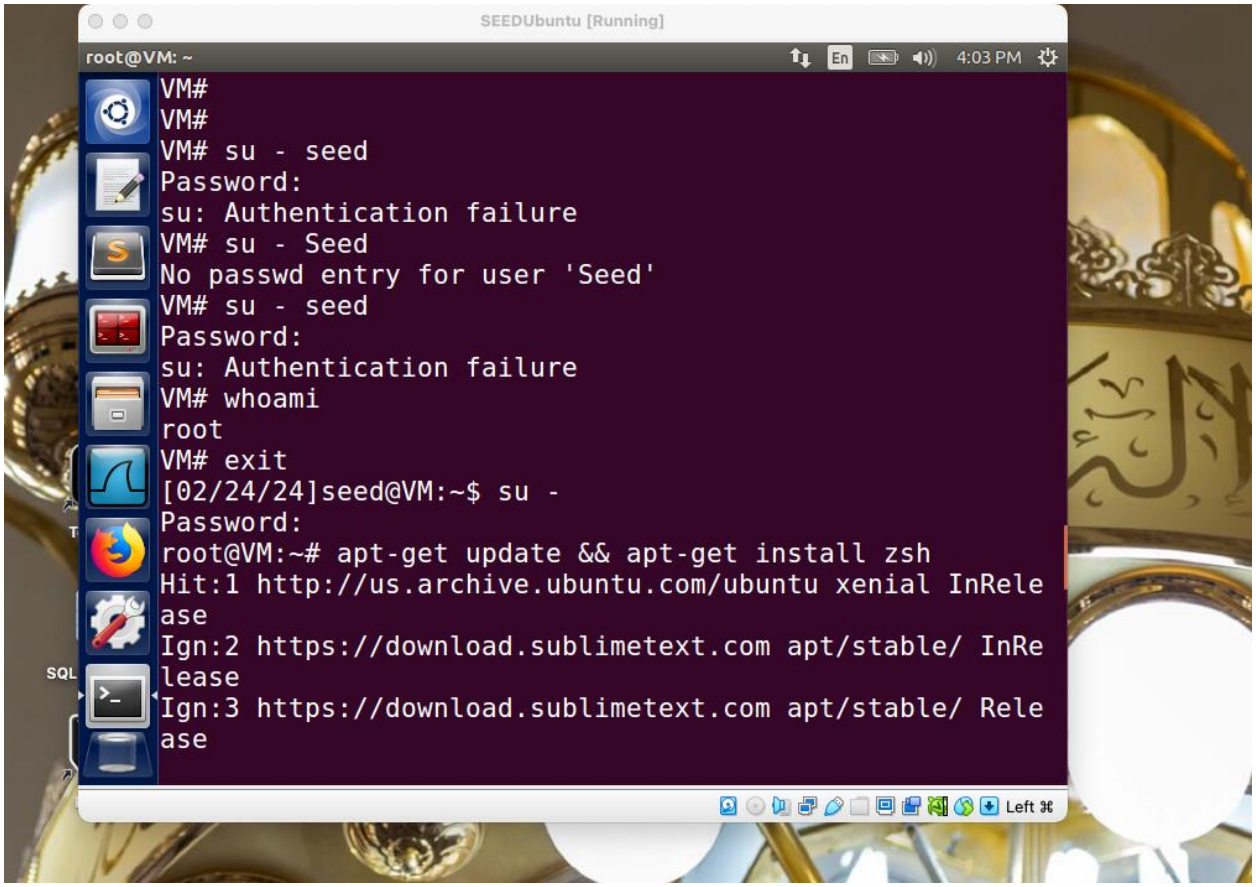
(a)

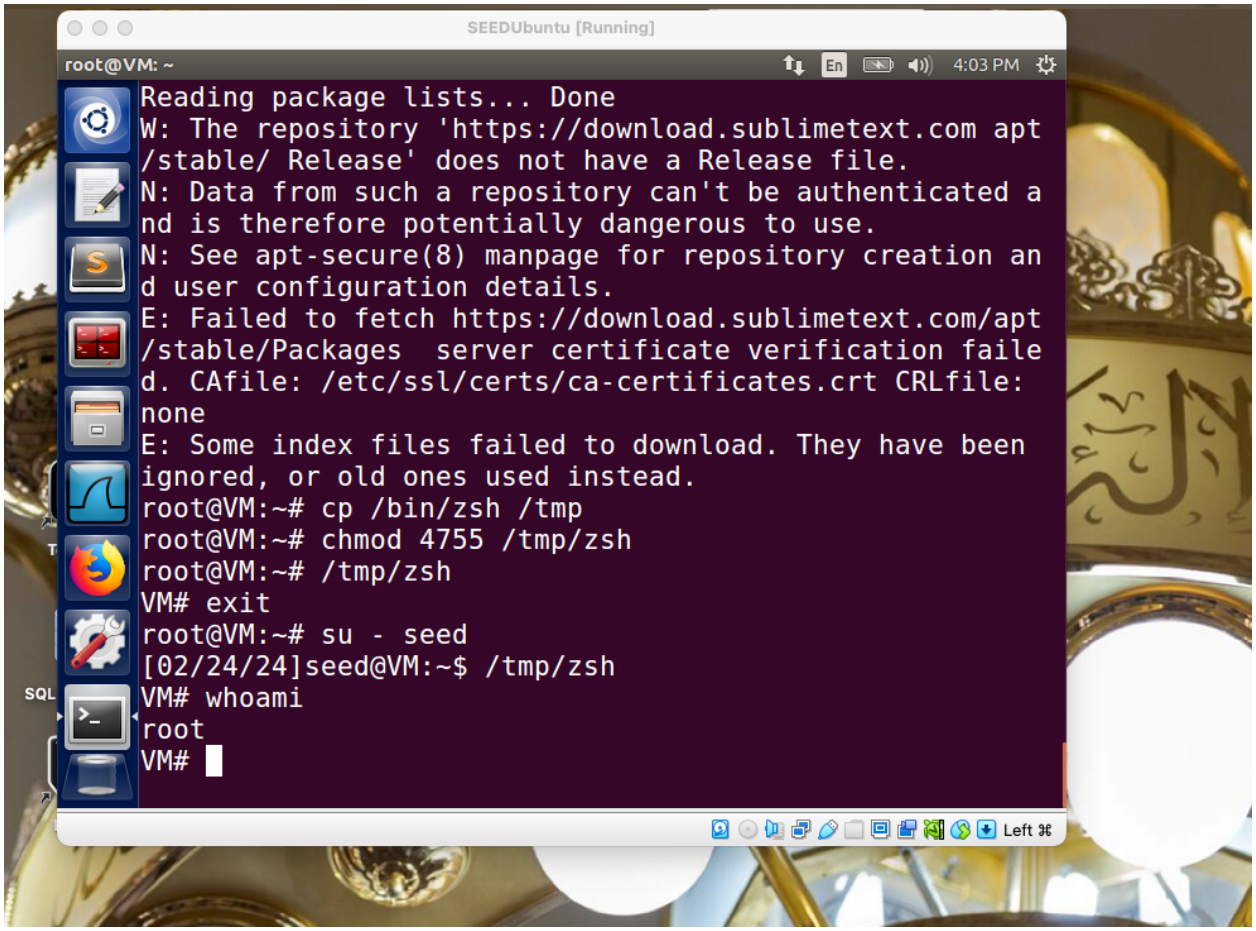
The "Authentication token manipulation error" typically occurs when the password-changing program encounters an issue updating the password. This can be due to several reasons, but in the context of my experiment, it is likely because the my_passwd command no longer has the Set-UID bit set, which would have allowed it to execute with root privileges. Without the Set-UID bit, the my_passwd command runs with regular user privileges, which do not have permission to write to the /etc/shadow file where the passwords are stored. As a result, the command cannot complete the action of updating the password, leading to the "Authentication token manipulation error." The expected behavior when the Set-UID bit is removed is that the command will fail to change the password due to insufficient permissions. My experiment successfully demonstrates and shows the importance of Set-UID for the proper functioning of commands that require elevated privileges, such as passwd.



(b1)

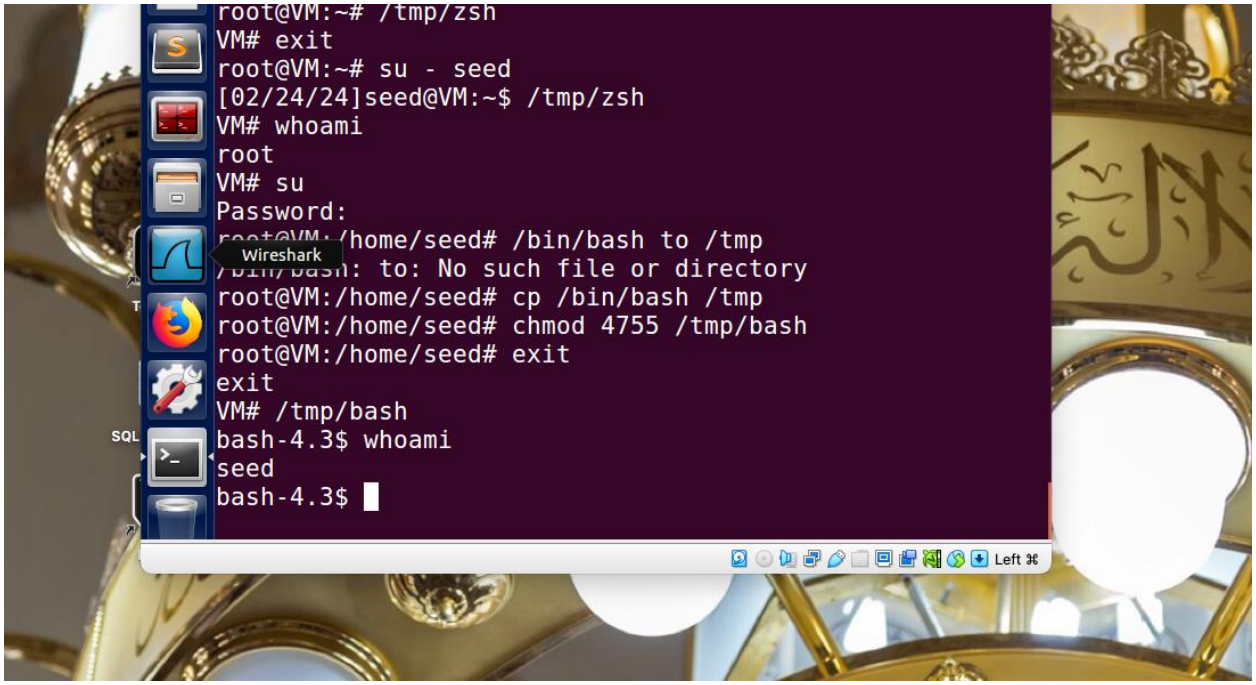
The outcome I observed confirms that the Set-UID bit worked as expected: the /tmp/zsh shell ran with the privileges of its owner, root, even when executed by a regular user. This is a significant security risk because any user able to execute /tmp/zsh could potentially perform actions as the root user, bypassing the normal security mechanisms that prevent regular users from obtaining such privileges. This behavior can vary on modern systems with enhanced security features. Some systems are configured to ignore the Set-UID bit on shell executables to prevent exactly this type of privilege escalation. However, based on my observation, it seems that the system does not have such protections in place, or they are not active, which is why I was able to obtain root privileges through the Set-UID zsh shell.





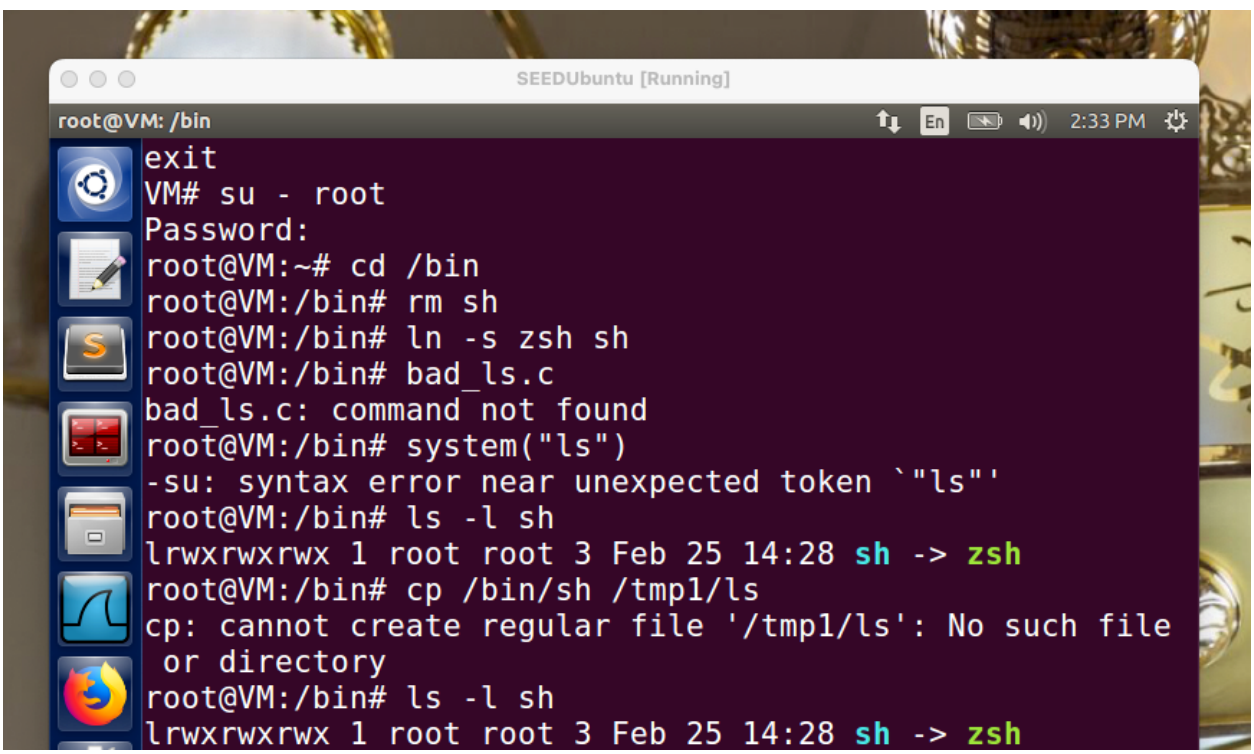
(b2)

Based on the prompt bash-4.3$ shown in my code screenshot, I did not get root privileges. When I ran a shell with root privileges, the typical prompt ends with a # symbol, not a $. The $ indicates that I am still operating as a regular user. If the shell had root privileges, the prompt would change to # after running /tmp/bash. This behavior suggests that the system is likely configured to ignore the Set-UID bit on shell programs for security reasons, as modern Unix-like systems often do. This is to prevent privilege escalation exploits where a user could gain root access through a Set-UID shell script or binary.

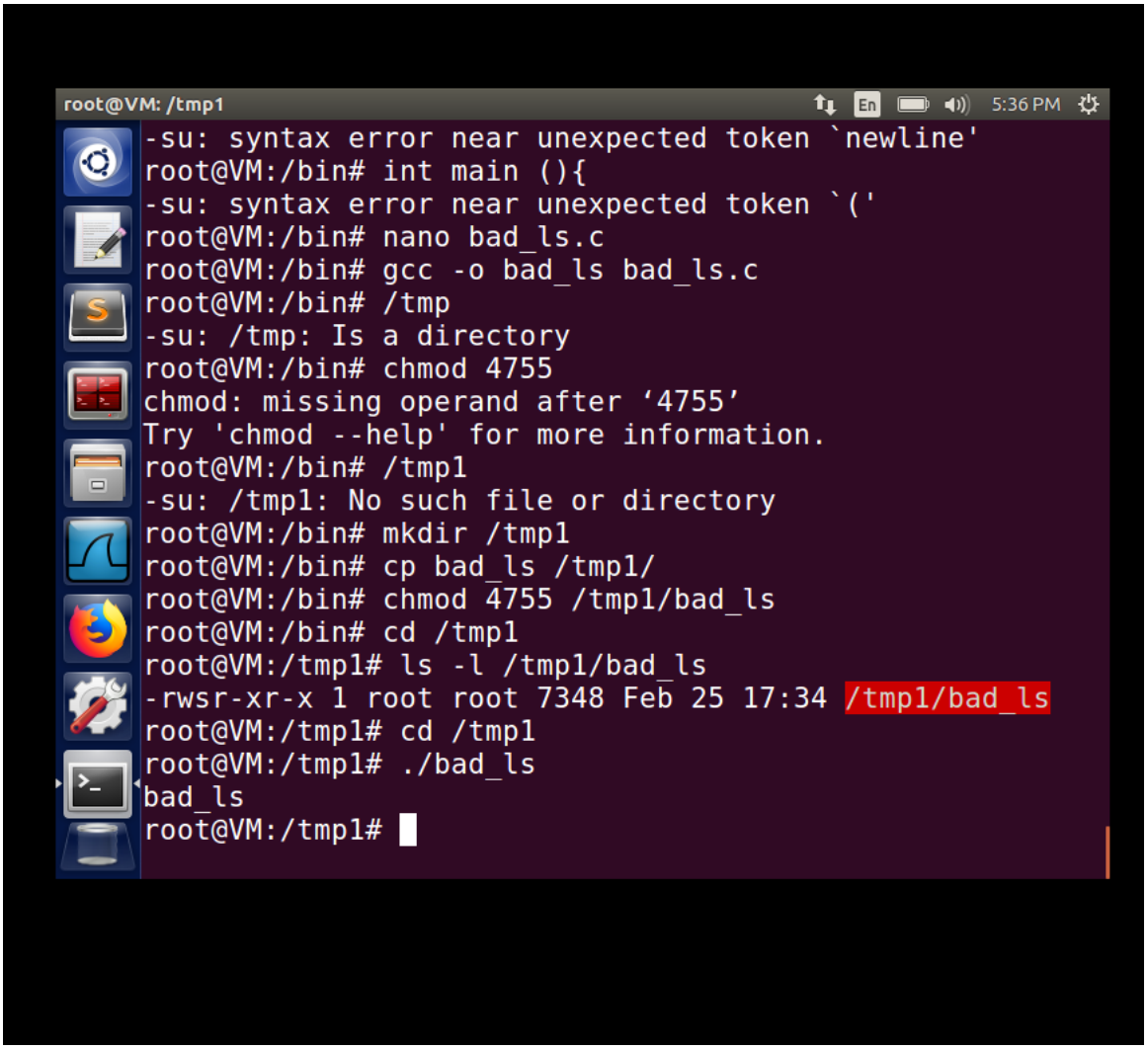


(c1)

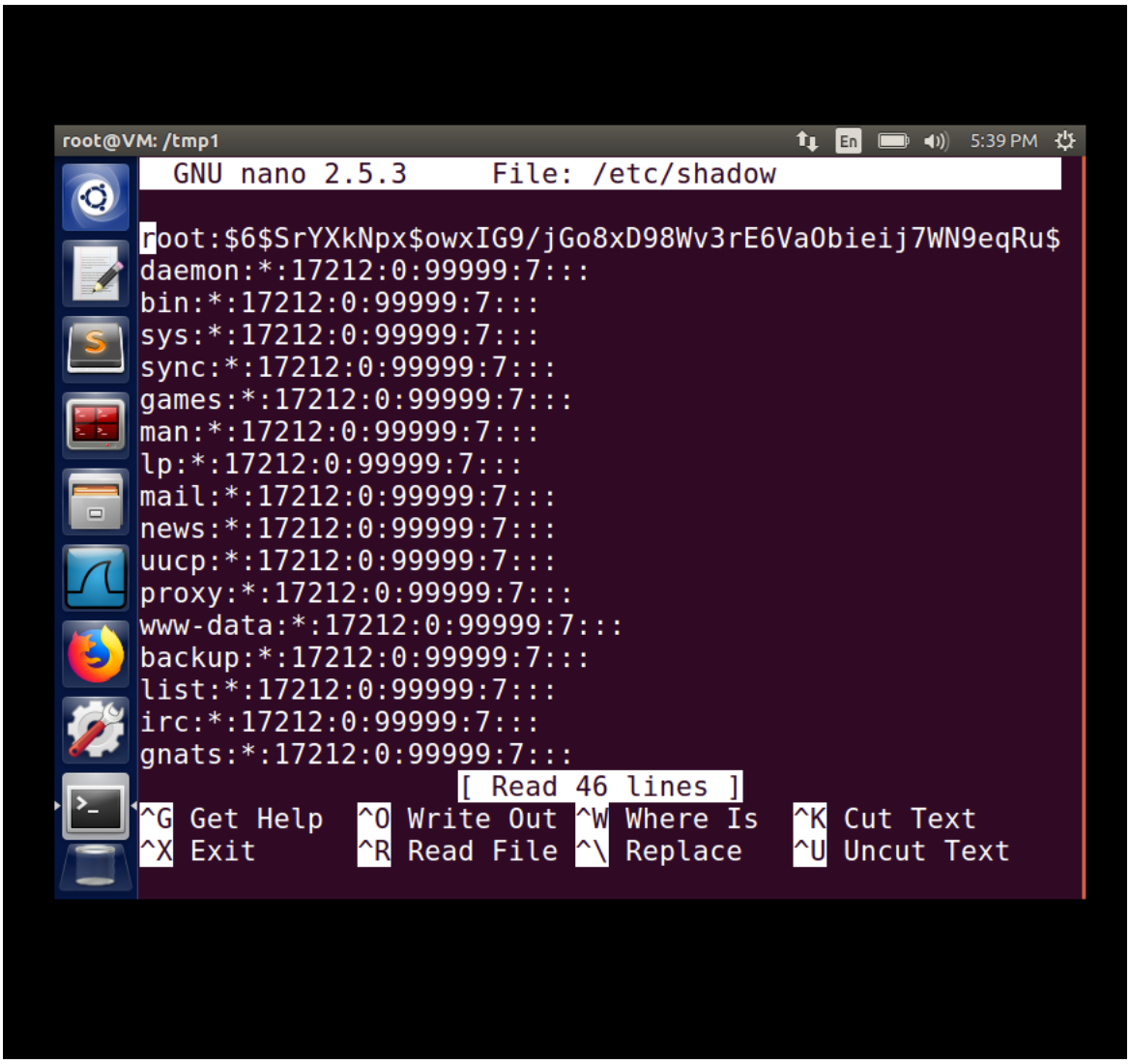
As shown below, we can see zsh has been set as the default shell.



Below, we can see that I was logged in as root, created a new directory /tmp1 and set it to have the same permissions as /tmp. Then I wrote this program into a file named bad_ls.c, compiled it (using gcc –o bad_ls bad_ls.c) and copied the executable as a Set-UID program into /tmp1 with permissions 4755.



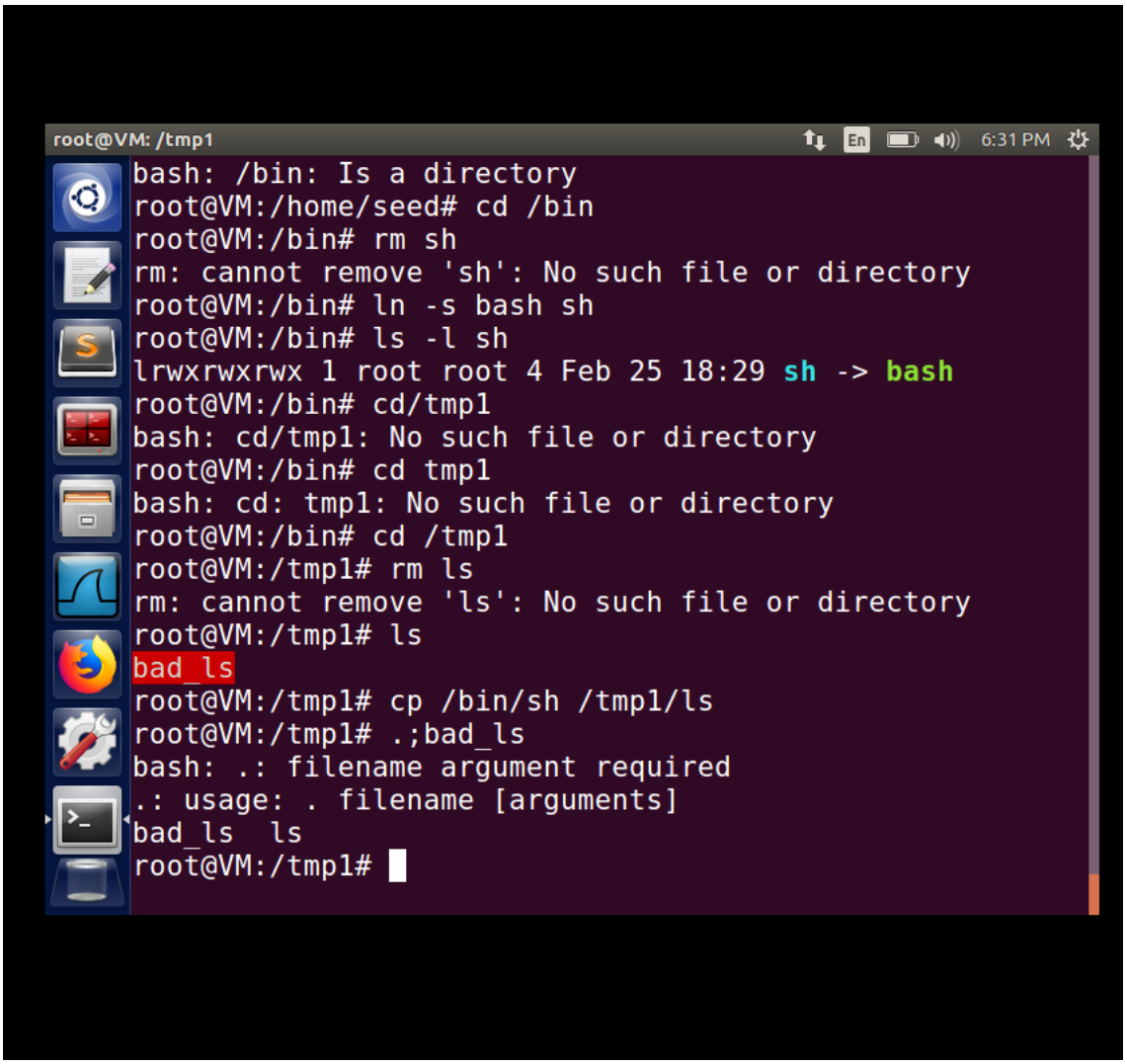
I then used the command nano /etc/shadow to open the file that contains the password information for the system's user accounts and is highly sensitive. It is only accessible by the root user due to its contents. As shown below, the user gets root privileges and can access the /etc/shadow file.



No, it is not a good idea to let regular users execute /tmp1/bad_ls if it is a Set-UID program owned by root. The reason being that allowing regular users to execute a Set-UID program like /tmp1/bad_ls, which is owned by root, poses a severe security risk. If this program internally calls other programs such as ls without specifying the absolute path, it is vulnerable to exploitation. For instance, an attacker could take advantage of this vulnerability to conduct a privilege escalation attack. The attacker would create a malicious script named ls, place it in a directory they control, and prepend that directory to the PATH environment variable. When the Set-UID bad_ls program is executed by any user, it would inadvertently run the malicious ls script with root privileges due to the altered PATH. This script could include commands to display or exfiltrate the contents of the /etc/shadow file, which contains sensitive password data. This kind of vulnerability underlines why developers should always use absolute paths in system calls within Set-UID programs and why system administrators should be cautious about the Set-UID permissions they grant, especially on scripts and executables that can be accessed by unprivileged users.

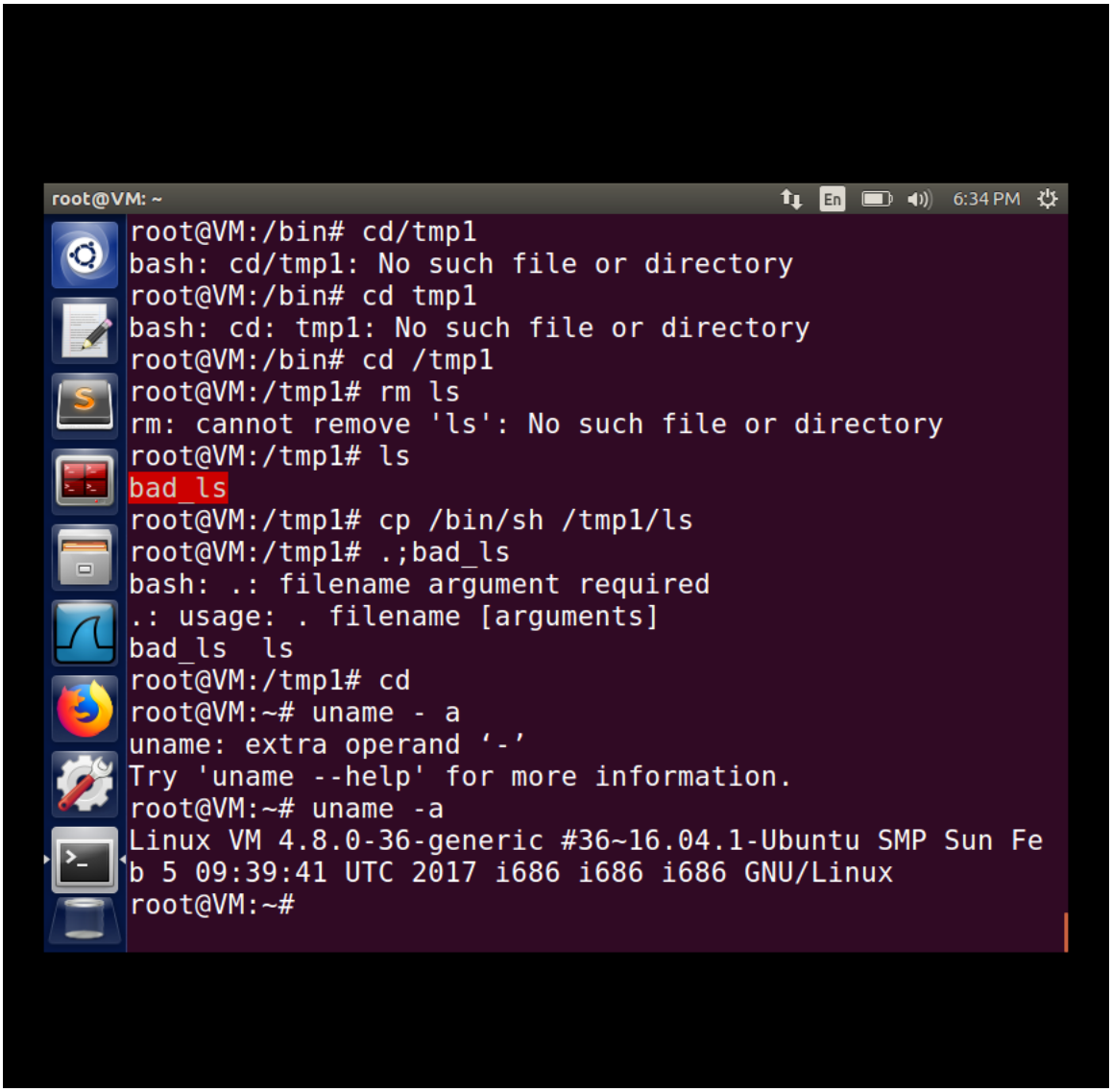
(c2)

As shown below, we did not get root privilege.



(c3)

As shown below, this is what Linux distribution you used for Problem 2 (distribution & kernel version).



Problem 3:

Based on the given scenario, there is a major flaw in how the security check only verifies using the visual means. An Intruder who is on the no-fly list can get into the flight following the steps described below:

1) The first thing the intruder is going to need is a fake government issued Identification. The photograph on the ID should closely resemble the person.
2) Next step would be to create a fake boarding pass. The individual can use online tools to create a fake boarding pass as an HTML page, there are many tools available on the internet today that let one accomplish this, including creating a two-dimensional barcode like a legitimate boarding pass. One can also use boarding pass templates to do this. Then the individual can print the fake pass.
3) Once the intruder has both the fake pass and the fake ID, he can proceed to the security check. Here, as mentioned, the major flaw in this system is that they only verify using visual inspection. The fake ID shows the image of the intruder, and the boarding pass has the same name as the fake ID so the intruder can get past the security check.
4) Now while boarding the flight, the boarding pass will be scanned, but it is to be noted that it would only check if the scanned information had an existing reservation on the flight, but it does not verify the identity of the passenger. If the system does not detect any issues with the scanned information, the intruder could potentially board the flight.

This way an Individual can easily bypass the airports security check in the scenario even when he is placed on the no-fly list.

To get past this security issue, we can implement the following:

1) Instead of using Visual Inspection, by using Biometric Verification like fingerprints or facial recognition during security checks and boarding, we could make sure that the individual matches their Identification and Boarding pass.
2) By enhancing the security on the boarding pass, like encryptions, we can make it harder for an individual to create a fake boarding pass. We can use digital signatures, etc. to avoid forging issues.
3) No-fly list should be integrated into the database for both the boarding pass and security checks, this way an individual would not be able to get past them easily.
4) Random checks should be done in the airport just in case an individual tries to bypass the security checks placed.